# Week 1. Neutron 구조와 SRE 관점

## 1. Neutron 구조

**K8s는 평평한 Pod 네트워크 하나를 주고, Neutron은 사용자가 network·subnet·router를 직접 만든다**

### 1-1. Neutron의 위치

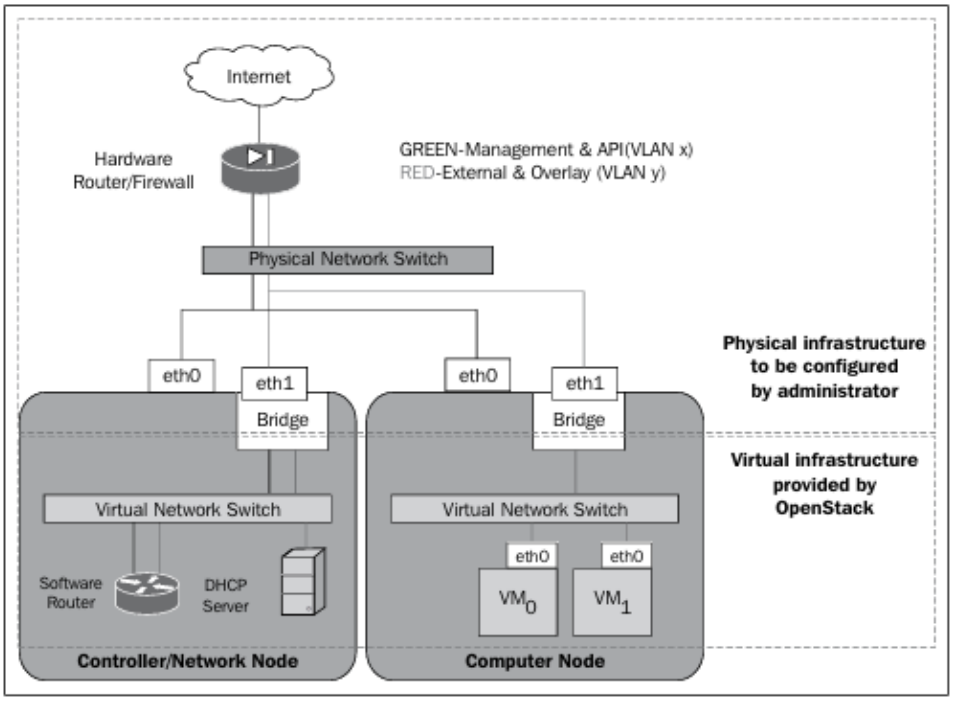
- **독립 서비스** → 따로 설치 가능, 여러 호스트에 나누거나 한 노드에 몰아서 운영 가능
- 요청 창구는 `neutron-server` → API로 받은 요청을 설정된 **network plugin**에 전달, 네트워크 설정은 **DB**에 저장
- server가 직접 스위치를 만드는 건 아니고, server가 DB에 저장한 정의를 plugin agent가 **RPC로 받아서** 가상 네트워크를 구성
- 사용자는 무엇을 연결할지를 API로 정의, 운영자는 어떤 기술로 구현할지(Linux bridge, OVS 등)를 선택

### 1-2. 제공 기능

| 기능 | 구현 | 비고 |
|---|---|---|
| Switching | Linux bridge, Open vSwitch(OVS) / VLAN, GRE·VXLAN 터널 | L2 가상 스위치 |
| Routing | IP forwarding + iptables + **network namespace** | namespace마다 라우팅 테이블, iptables 분리 |
| Load balancing | LBaaS | Neutron LBaaS는 Queens에서 deprecated, 현재는 Octavia가 대체 |
| Firewalling | 보안 그룹(compute 노드의 iptables), FWaaS(라우터에서 처리) | |
| VPN | IPSec 기반 VPNaaS | |

- namespace가 네트워크 스택을 통째로 분리해 주기 때문에 **테넌트끼리 같은 IP 대역을 써도 충돌하지 않음**

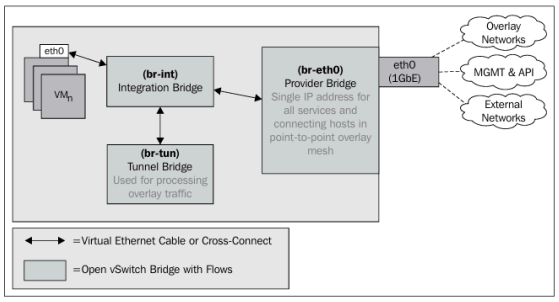
### 1-3. 트래픽 네 종류

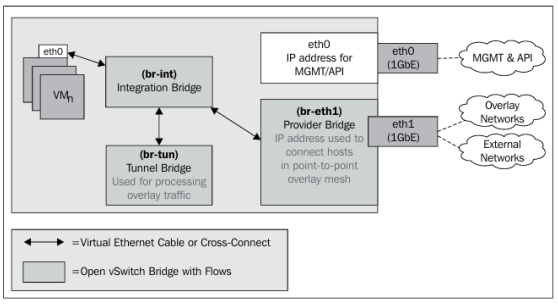



| 구분 | 용도 | 비고 |
| --- | --- | --- |
| Management | 호스트 간 내부 통신 (메시징, DB 등) | 모든 호스트가 참여 |
| API | 사용자·서비스가 OpenStack API에 접근 | Keystone, Neutron, Glance, Horizon 엔드포인트 |
| External | Neutron 라우터가 바깥으로 나가는 길 | |
| Guest | 인스턴스 트래픽 전용 | local / flat / VLAN / GRE·VXLAN |

- 인터페이스 네 개 X → 한 인터페이스에 합쳐도 됨

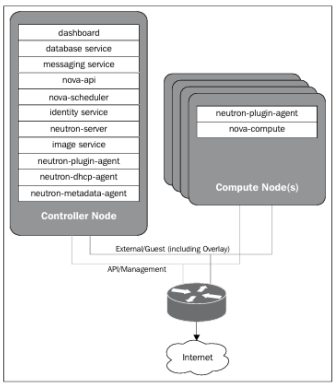
| NIC 구성 | 특징 |
| --- | --- |
| 1개 | 모든 트래픽이 한 인터페이스 → 게스트가 대역폭을 독점하면 호스트까지 영향, **테스트·PoC 전용** |
| 2개 이상 | (관리+API) / (외부+게스트)로 분리, 운영 권장 구성 |
| Bonding | 링크를 묶어 대역폭 확장 또는 active/passive 이중화, 물리 스위치 지원 필요 |

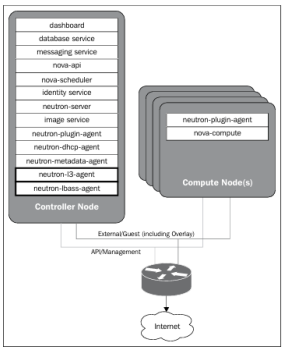


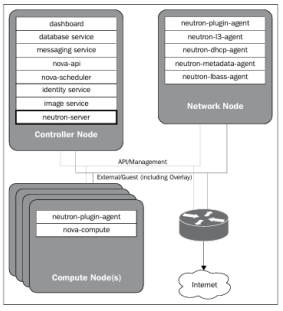




## 1-4. 서비스를 노드에 나누는 방식

```
[Controller]    neutron-server · DB · MQ · Keystone · Glance · nova-api ...
[Network node]  l3-agent · dhcp-agent · metadata-agent · plugin agent   (선택)
[Compute]       nova-compute · plugin agent
```




| 모델 | 구성 | 특징 |
|---|---|---|
| ① controller + compute (L3 없음) | controller에 neutron-server, dhcp, metadata, plugin agent | Neutron은 L2만 제공 |
| ② controller + compute (L3 있음) | ①에 l3-agent 추가 | 소프트웨어 라우터가 controller에서 동작 |
| ③ controller + network node + compute | neutron-server만 controller, 나머지 agent는 network node | controller가 네트워크 포화에서 분리 → 보안, 복원력 향상 |

- **L3 agent가 없으면 Neutron은 L2만 제공 → 세그먼트 간 라우팅은 외부 물리 라우터 몫**

## 1-5. 달라진 점 (공식 문서 참고)

| 책 | 현재 |
|---|---|
| `neutron` CLI | 제거됨 → `openstack network ...` openstack 명령 사용 |

| 책 | 현재 |
| --- | --- |
| LBaaS (HAProxy agent) | Octavia로 대체 (neutron-lbaas는 Queens에서 deprecated) |
| monolithic plugin (LinuxBridge / OVS 중 택 1) | ML2가 여러 L2 기술(OVS, OVN, SR-IOV, MacVTap 등)을 함께 쓰는 구조 |
| dhcp-agent, l3-agent | **OVN 배포에서는 OVN이 대체** → 해당 agent 프로세스 없음 |
| metadata-agent | OVN은 각 hypervisor의 `neutron-ovn-metadata-agent`가 담당 (namespace 안 haproxy → UNIX socket → agent → Nova) |

- 보안 그룹은 port 단위로 적용, stateful / 구현은 iptables 외에 OVS 네이티브 방화벽 드라이버도 있음
- **ML2:** 여러 L2 기술을 동시에 쓰게 해 주는 플러그인
- **OVN:** DHCP와 L3 처리를 직접 맡아 **neutron-dhcp-agent와 neutron-l3-agent를 대체**
- **클러스터가 ML2/OVS인지 OVN인지에 따라 장애 추적 지점이 달라짐** → `openstack network agent list`로 확인 필요

# 2. 기본 구성 요소와 Agent

## 2-1. Network / Subnet / Port

| 리소스 | 정의 | 포인트 |
| --- | --- | --- |
| Network | 격리된 L2 브로드캐스트 도메인 | 기본은 만든 테넌트 전용(공유 가능), **subnet·port는 반드시 network에 속함** |
| Subnet | IPv4/IPv6 주소 블록 | **CIDR 필수**, 한 network에 여러 개 가능(비연속 가능), DHCP 할당 범위 설정 가능 |
| Port | 논리 가상 스위치의 포트 | VM의 NIC가 매핑, **MAC·IP 결정**, DB에 저장 → RPC로 받아 구성 |

```
사용자:       network / subnet 생성
Nova:         VM 인터페이스를 port에 연결
plugin agent:  RPC로 받아 노드에 가상 스위치 구성
```

- 플러그인이 바뀌어도 사용자가 보는 API는 동일

## 2-2. network namespace

| 이름 | 역할 | 비고 |
| --- | --- | --- |
| `qdhcp-<network UUID>` | 해당 network의 DHCP 서비스 | 가상 스위치에 인터페이스 연결 |
| `qrouter-<router UUID>` | 라우터 하나 | 연결된 subnet 간 라우팅, NAT |

- openstack은 멀티테넌시 전제 → 테넌트마다 private network, router, firewall, LB 여러 개 가질 수 있어야 함
- namespace는 라우트, 방화벽 규칙, 인터페이스를 가진 **네트워크 스택의 논리 복사본** → **서로 다른 테넌트가 같은 IP 대역을 써도 됨**
- ML2/OVS 기본 구성에서는 **controller나 network node**에 있음
- 목록: `ip netns` / namespace 안에서 실행: `ip netns exec <NAME> <command>` (`ip`, `route`, `iptables` 등)

## 2-3. Agent 역할

| Agent | 역할 | 메모 |
| --- | --- | --- |
| `neutron-server` | API 수신 → plugin에 전달 | plugin을 지정하지 않으면 시작 불가 |
| plugin agent | 각 노드에서 가상 스위칭 구성 (linuxbridge / openvswitch) | controller, compute 모두 필요 |
| `neutron-dhcp-agent` | DHCP를 쓰는 네트워크마다 **dnsmasq** 프로세스 생성·관리, metadata proxy도 함께 생성 | `qdhcp-*` namespace에서 동작 |
| `neutron-metadata-agent` | 인스턴스의 메타데이터 요청을 Nova metadata API로 중계하는 프록시 | Nova와 shared secret 공유 해야 동작 |
| `neutron-l3-agent` | 가상 라우터, Floating IP | Linux IP 스택 + iptables로 L3 forwarding, NAT |

- **서버는 정의를 받아 저장, 실제 구성은 노드마다 도는 agent가 담당**
  - 전체 네트워크 구조는 DB에 있고, 각 노드의 plugin agent가 그대로 가상 네트워크를 만들며, DHCP·metadata 서비스가 부팅 때 IP와 인스턴스 정보를 전달한다.
- OVN 기반 배포에서는 DHCP와 L3를 OVN이 대신하므로 `neutron-dhcp-agent`, `neutron-l3-agent` 프로세스가 없음
  - 메타데이터는 `neutron-ovn-metadata-agent`가 각 하이퍼바이저에서 맡고, namespace 안의 haproxy가 요청을 UNIX socket으로 이 agent에 넘기고 agent가 Nova로 전달

## 2-4. 설정 옵션

| 옵션 | 의미 | 주의 |
| --- | --- | --- |
| `core_plugin` | 사용할 L2 plugin 지정 | |
| `dhcp_lease_duration` | DHCP 임대 시간 | 기본 86400초(24h), `-1` 이면 무한 **(IP 점유 기간X)** |
| `bind_host` | API가 listen하는 주소 | 기본 `0.0.0.0` → 관리/API 네트워크로 제한 권장 |
| `allow_overlapping_ips` | 테넌트 간 서브넷 겹침 허용 | 기본 True, 커널의 namespace 지원 필요 |
| `interface_driver` | agent가 인터페이스를 만드는 방식 | plugin에 맞춰 하나만 지정, 기본값은 `openvswitch` |
| `enable_isolated_metadata` | router가 없는 subnet에서도 DHCP로 메타데이터 경로 제공 | 기본 False, 게스트가 DHCP option 121을 요청해야 동작, `force_metadata` 켜면 전체 적용 |
| `enable_metadata_network` | agent가 라우터와 다른 호스트에 있을 때 169.254.0.0/16 대역 subnet을 메타데이터 전용 network로 취급 | `enable_isolated_metadata = True` 필요 |

- `dhcp_lease_duration` **은 IP를 갖고 있는 기간이 아님** → 한 번 할당된 IP는 인스턴스나 port를 삭제하기 전까지 유지 (꺼져 있어도)

## 2-5. 메타데이터 요청 흐름

인스턴스가 부팅하며 자기 정보를 가져올 때

1. 인스턴스가 부팅 중 `169.254.169.254` 로 HTTP 요청을 보낸다.
2. 경로에 따라 라우터 또는 DHCP namespace에 요청이 도착한다.
3. 그 안의 metadata proxy가 Unix socket으로 neutron-metadata-agent에 전달한다.
4. metadata agent가 Nova metadata API로 전달한다.
5. Nova metadata API가 응답을 agent에 돌려준다.
6. agent가 응답을 proxy에 보낸다.
7. proxy가 HTTP 응답을 인스턴스에 전달한다.
8. 인스턴스가 정보를 처리하고 부팅을 계속한다.

- 메타데이터 경로가 끊기면 hostname, 공개키 같은 부팅 시 정보를 못 받음
  → **부팅은 되는데 키가 안 들어감**(=Pod는 Running인데 접속X)

# 3. SRE(Monitoring Distributed Systems)

## 3-1. 용어

| 용어 | 의미 |
|---|---|
| White-box | 시스템 내부 지표(로그, 내부 통계 핸들러 등) 기반 |
| Black-box | 사용자가 보는 외부 동작을 직접 시험 |
| Dashboard | 서비스 핵심 지표를 모아 보여 주는 화면 |
| Alert | 사람이 읽도록 보내는 알림 → ticket / email / **page** |
| Root cause | 고치면 같은 방식으로 재발하지 않는다고 볼 수 있는 결함 (한 사건에 여러 개 가능) |

- page는 사람의 시간을 쓰는 비싼 수단 → 잦으면 알림을 무시하게 되고 진짜 page가 묻힘
- 좋은 알림 체계 = **신호는 최대, 소음은 최소**

## 3-2. 증상 vs 원인

| 증상 (what) | 원인 (why) |
|---|---|
| HTTP 500/404 응답 | DB 서버가 연결을 거부 |

| 증상 (what) | 원인 (why) |
| --- | --- |
| 응답이 느림 | CPU 과부하, 또는 케이블 불량으로 인한 부분적 패킷 손실 |
| 특정 지역 사용자만 콘텐츠 못 받음 | CDN이 일부 클라이언트 IP를 차단 |
| 비공개 콘텐츠가 모두에게 보임 | 새 배포로 ACL이 빠짐 |

- **무엇이 고장났나(증상)에 page를 걸고, 왜(원인)는 디버깅 보조로 사용**

| 구분 | 보는 것 | 강점 | 한계 |
| --- | --- | --- | --- |
| Black-box | 사용자 관점의 외부 동작 | 이미 진행 중인 증상만 잡아 page 규율 유지 | 임박한 문제는 못 잡음 |
| White-box | 내부 지표 | 임박한 문제, 재시도에 가려진 실패 감지, 디버깅에 필수 | 증상이면서 원인일 수 있어 해석 필요 |

- 웹 서버가 느릴 때 → **"web이 본 DB 속도"와 "DB가 본 자기 속도"**를 둘 다 봐야 DB 문제인지 사이 네트워크 문제인지 구분

## 3-3. Four Golden Signals



| 신호 | 의미 | 주의 |
| --- | --- | --- |
| Latency | 요청 처리 시간 | 성공/실패 지연을 분리, 느린 에러가 빠른 에러보다 더 나쁨 |
| Traffic | 시스템에 걸린 수요 | 서비스에 맞는 지표 (웹은 초당 요청 수) |
| Errors | 실패한 요청의 비율 | 명시적(500) / 암묵적(200인데 내용 틀림) / 정책(약속한 시간 초과) |
| Saturation | 얼마나 찼는지 | 가장 제약된 자원 기준, 100% 전에 성능이 떨어짐, 지연 증가가 선행 지표 |

## 3-4. Tail Latency

- 평균 100ms, 초당 1,000건인데 1%가 5초일 수 있음
- 지연 값 자체 대신 **구간별 요청 수**를 수집, 경계는 약 3배씩 (0–10, 10–30, 30–100, 100–300ms ...)

| | |
| --- | --- |
| **White-box** | 각 컴포넌트가 `/metrics` 로 내보내는 내부 지표를 **Prometheus**가 수집하는 방식입니다. |
| **Black-box** | **Prometheus Blackbox Exporter**처럼 바깥에서 HTTP나 TCP로 직접 찔러 보는 방식입니다. |
| **page** | **Alertmanager**가 사람에게 보내는 호출입니다. 책의 기준대로 사용자가 보는 에러율과 지연(증상)에 걸고, 노드 CPU 같은 원인은 대시보드에서 봅니다. |

## 3-5. page를 만들기 전 확인할 것

1. 다른 방법으로 못 잡는 상태이면서 **긴급 / 조치 가능 / 사용자에게 보이는** 문제인가
2. 이 알림을 무해하다고 무시할 때가 있는가 → 있다면 없애는 방법은
3. 사용자가 실제로 피해를 입고 있는가 걸러야 할 경우(트래픽 drain, 테스트 배포 등)는?
4. 내가 조치할 수 있는가, 긴급한가, 자동화 가능한가, 장기 해결인가 임시 우회인가
5. 같은 문제로 다른 사람도 호출되어 page 하나가 불필요하진 않은가

## 3-6. 사례

| 사례 | 문제 | 대응 |
| --- | --- | --- |
| Bigtable | SLO가 평균 기반인데 최악 5% 요청이 느려서 이메일·page 폭주, 정작 사용자 영향 문제는 놓침 | 성능 개선 + SLO를 75퍼센타일로 일시 완화 + 이메일 알림 끔 → 근본 문제를 고칠 여유 확보 |
| Gmail | Workqueue에서 task가 de-schedule될 때마다 알림, task가 수천 개라 유지 불가 | 스케줄러를 찔러 주는 도구 제작, 전체 자동화가 근본 수정을 늦출지 논쟁 |

- **VM → 브리지 → 터널 → 라우터 → 보안 그룹으로 패킷 경로를 따라 내려가며 범위 좁혀가기**

# 결론

> Neutron은 network·subnet·port를 DB에 정의하고 각 노드의 agent가 이를 실제 네트워크로 만들며, 장애 추적은 증상(what)에서 시작해 원인(why)으로 한 층씩 좁혀 가는 일

### Q1. Neutron은 요청을 어떻게 실제 네트워크로 만드는가

- `neutron-server`가 API 요청을 받아 plugin에 전달, 정의는 DB에 저장
- 각 노드의 plugin agent가 DB의 network / subnet / port 정의를 RPC로 받아 가상 스위치를 구성
- Nova가 VM 인터페이스를 port에 연결 → port가 MAC·IP를 결정

### Q2. agent는 각자 무슨 일을 하고 어디에 놓이는가

- dhcp-agent: dnsmasq + `qdhcp` namespace, l3-agent: 라우터·Floating IP + `qrouter` namespace, metadata-agent: 메타데이터 중계
- 배치는 controller / network node / compute로 나누고, network node가 있으면 agent를 그쪽으로 이동
- namespace 덕분에 테넌트별 격리와 IP 대역 중복이 가능
- OVN 배포는 dhcp / l3 agent를 OVN이 대체

### Q3. 인스턴스의 메타데이터 요청은 어떤 경로로 가는가

- `169.254.169.254` → router 또는 DHCP namespace의 proxy → Unix socket → metadata-agent → Nova metadata API
- 응답은 같은 경로를 거꾸로 돌아옴
- 이 경로가 끊기면 부팅 시 hostname, 공개키 같은 정보를 못 받음

**Q4. 모니터링에서 무엇에 page를 걸어야 하는가**

- 원인보다 **증상** 기준, 긴급하고 조치 가능하고 판단이 필요한 새로운 문제에만
- Four Golden Signals(latency, traffic, errors, saturation)를 보고, 평균이 아니라 꼬리를 봄
- 기계적으로 반복되는 page는 자동화·근본 해결 대상, 이메일 알림은 dashboard로 대체